

**"PrimeBid" - Forward Auctioning System**

**EECS 4413: Building e-Commerce Systems**

**Software Design Document - Requirements  
Documentation**

**Team 5: Ascendants**

**Submission Date: February 6, 2026**

## **Document Change Control:**

| Version | Date       | Author(s)                          | Summary Of Changes                                                                                                                                                           |
|---------|------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.1     | 2026-02-03 | Nathan Binu Edappilly<br>219317965 | Initial creation of Software Design Document. Added project overview, 2 Activity diagrams for UC1 and UC2, and initial design decisions. Developed the Architecture section. |
| 0.2     | 2026-02-03 | Umer Shaikh<br>218 931 790         | Created Sequence Diagrams for UC1, UC2, and UC3. Completed remaining Activity Diagrams (UC4–UC7) to ensure full use case coverage.                                           |
| 0.3     | 2026-02-03 | Imaduddin Ahmed<br>219 569 029     | Designed the UML Component Diagram and defined the Service Interfaces and API specifications.                                                                                |
| 0.4     | 2026-02-03 | Pawanjot Randhawa<br>219 732 718   | Added Project Plan components: Product Backlog, GANTT Chart, and Group Meeting Logs. Compiled the TDD (Test Driven Development) section.                                     |
| 1.0     | 2026-02-05 | All Members                        | Final integration of all sections. Added AI Acknowledgment and References. Verified formatting and diagram consistency for final submission.                                 |

# **Table of Contents**

1. Main Page
2. Introduction:
  - a. 2.1 Purpose
  - b. Overview
3. Major Design Decision
4. Sequence Diagrams
5. Activity Diagrams
6. Architecture
7. Activities Plan, Product Backlog, and Sprint Backlog
8. Test Driven Development (TDD)
9. Acknowledgment
10. References

## **2. Introduction**

### **2.1 Purpose**

The purpose of this Software Design Document (SDD) is to describe the structure, behavior, and key design decisions of the Forward Auctioning System. This document serves as a reference for developers, testers, and evaluators **by** clearly outlining the system requirements, architecture, and design constraints. It supports the implementation and validation of the system across all project milestones.

### **2.2 Overview**

The Forward Auctioning System is a web-based application that enables users to participate in forward (highest-bidder) auctions. The system allows users to register, authenticate, browse auctioned items, place bids in real time, and complete payments if they win an auction. Sellers are able to upload items for auction by specifying item details, auction duration, and starting bid price.

The system enforces the following key rules and constraints:

- Only forward auctions are supported.
- All bids must be integer values and strictly higher than the current highest bid.
- Users may bid on only one item per login session.
- Only the winning bidder may proceed with payment once an auction ends.
- Final payment includes item price, shipping cost, and optional expedited shipping.
- Receipt and shipment information are displayed after payment confirmation.

The system is designed using a modular and service-oriented architecture, enabling clear separation of concerns between user management, auction handling, payment processing, and supporting services. This document provides

an executive summary of the system requirements and the design approach used to implement them in a maintainable and extensible manner.

## 3. Major Design Decisions

### 3.1 Technology Stack:

1. **Front-End (Presentation Layer):** We use **HTML and CSS**, with optional **TypeScript**, to implement the web-based user interface and basic client-side interactions.
2. **Gateway and Application Layer:** We use **Java with the Spring Boot framework** to implement the Gateway (Façade) component and coordinate application workflows through RESTful APIs.
3. **Back-End Services (Business Logic Layer):** We use **Java and Spring Boot** to implement backend services, including IAM, Catalogue, Auction, Payment, and an Advanced Feature Service planned for Milestone 3.
4. **Data Layer:** We use **SQLite** as a lightweight relational database for persistence, with each service managing its own database to maintain low coupling and clear data ownership. **JDBC** is used for database access.
5. **Communication:** We use **RESTful HTTP with JSON payloads** for service communication. **WebSocket communication** may be used to support real-time bid updates.
6. **Deployment (Milestone 3):** We plan to use **Docker containers** to support service isolation and deployment for the advanced milestone.

### **3.2 Modularization and System Decomposition**

The system is designed using a modular architecture to ensure high cohesion and low coupling between components. Each major responsibility of the Forward Auctioning System is encapsulated within a distinct module:

- **User Authentication Module:** Handles user sign-up, sign-in, password verification, and session validation.
- **Catalogue Module:** Manages auction items, including item descriptions, keywords, shipping costs, and auction durations.
- **Auction Module:** Controls the bidding process, enforces auction rules, tracks bids and bidders, and determines auction outcomes.
- **Payment Module:** Handles payment processing, validation of required payment details, and receipt generation.
- **User Interface Module:** Provides the web-based interface through which users interact with the system.

This separation ensures that changes in one module (e.g., payment processing) do not directly impact other modules (e.g., bidding logic), improving maintainability and testability.

### **3.3 Architectural Pattern Chosen**

The system follows a **Layered Architecture**, consisting of the following layers:

1. **Presentation Layer:** User-facing pages for registration, login, browsing auctions, bidding, payment, and receipt display.
2. **Application Layer:** Controllers and services that coordinate use case execution and enforce system rules.
3. **Domain Layer:** Core business logic, including auction rules, bidding validation, and payment conditions.
4. **Data Layer:** Databases responsible for storing users, auction items, bids, and payment records.

This architecture improves maintainability, supports clear separation of concerns, and aligns with standard web application design practices taught in the course. A **Gateway service** is used as a Façade between the user interface and backend modules. It provides a single entry point for UI requests,

orchestrates use case flows, and enforces system-wide constraints such as allowing users to bid on only one item per login session.

### **3.4 Architectural Pattern Considered but Not Chosen**

A **fully distributed Microservices Architecture** was considered but not selected for this deliverable. While microservices offer benefits such as independent deployment and scalability, they introduce additional complexity in terms of inter-service communication, data consistency, and deployment infrastructure.

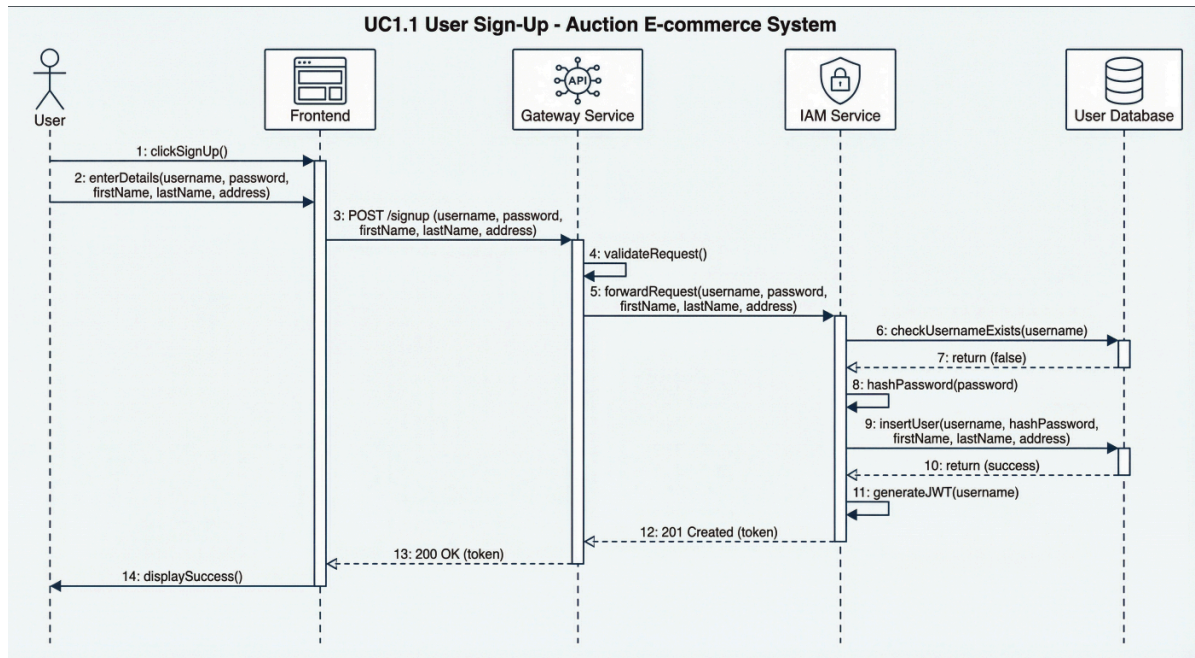
Given the academic scope of the project and the objectives of the early milestones, a layered modular architecture provides sufficient flexibility and clarity without unnecessary overhead. Elements of service-based and event-driven communication are reserved for later milestones as optional extensions.

### **3.5 Design Constraints and Assumptions**

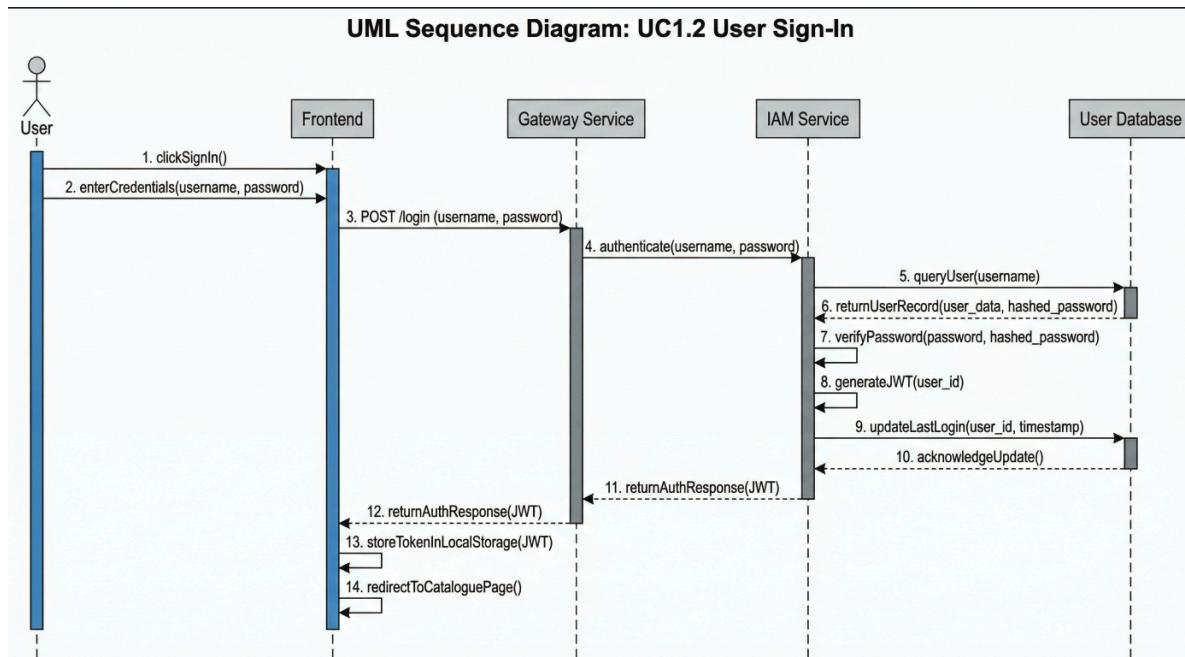
- The system uses server-side page refreshes rather than real-time push notifications.
- Payment details entered during testing are not real credit card data.
- Shipping dates are displayed as textual information (e.g., “Item will be shipped in XXX days”).
- Advanced AI-assisted or self-adaptive features are deferred to **Milestone 3** and are not implemented in earlier milestones.

## 4. Sequence Diagrams

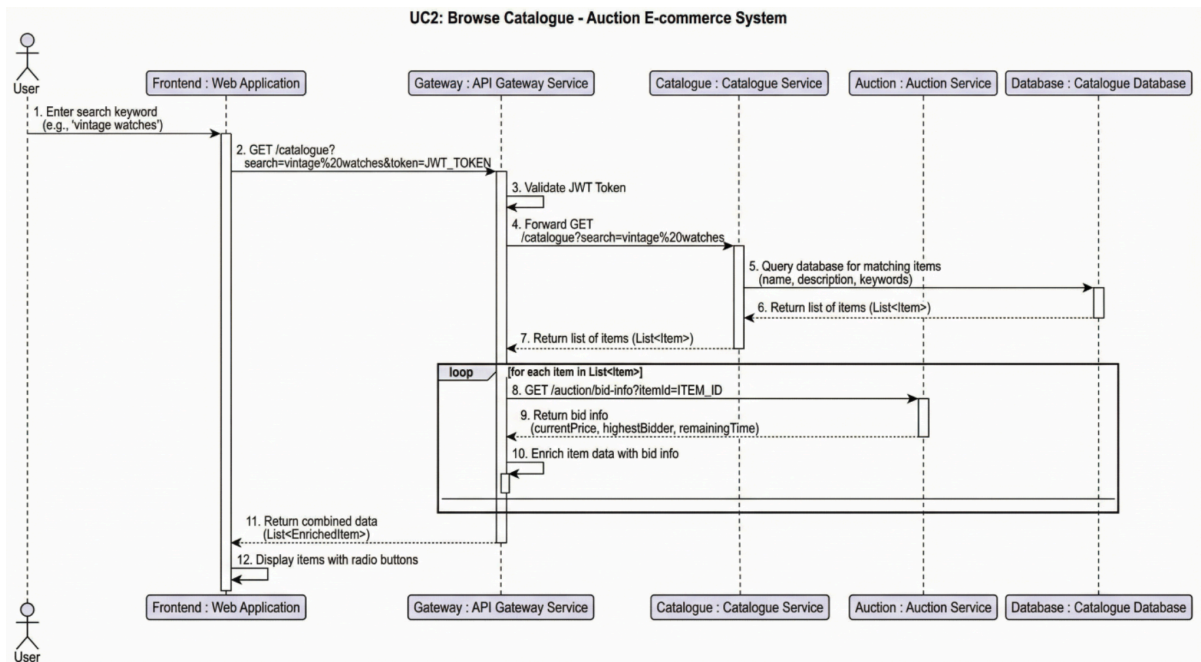
### Sequence diagram for UC1.1 - Sign-Up:



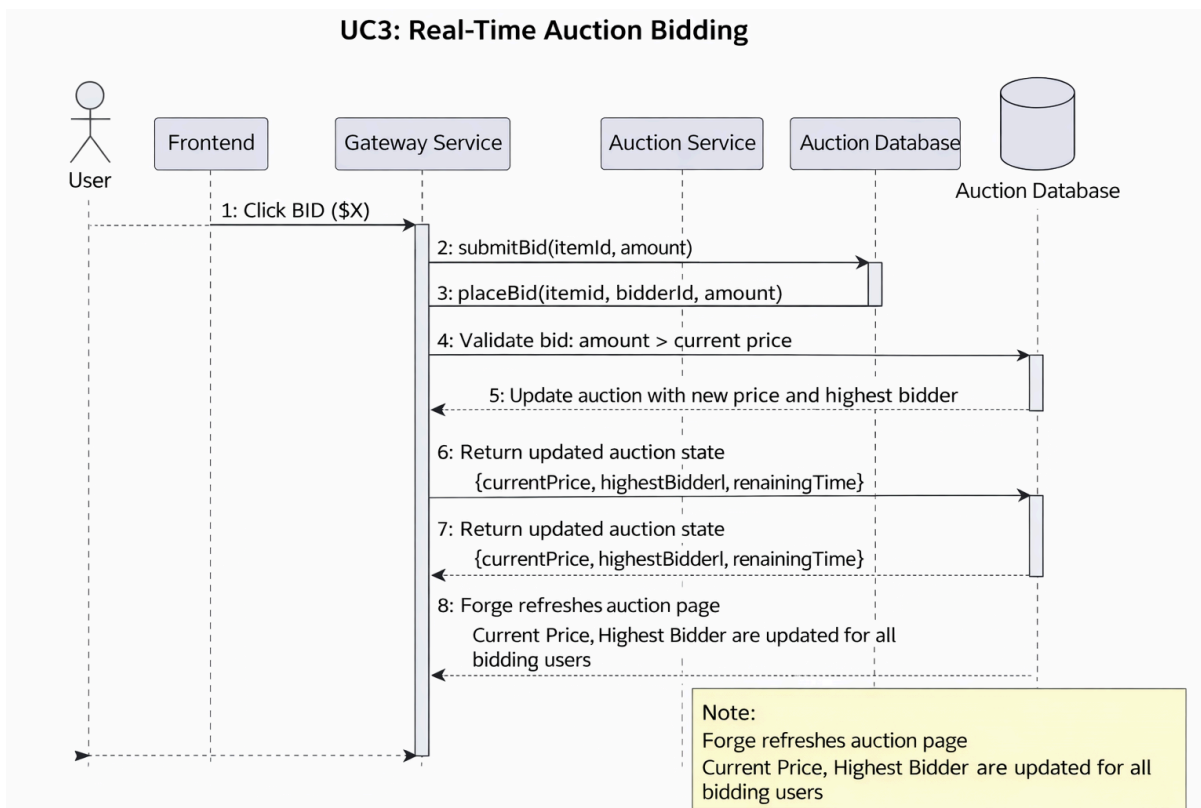
### Sequence diagram for UC1.2 - Sign-In:



Sequence diagram for UC2:



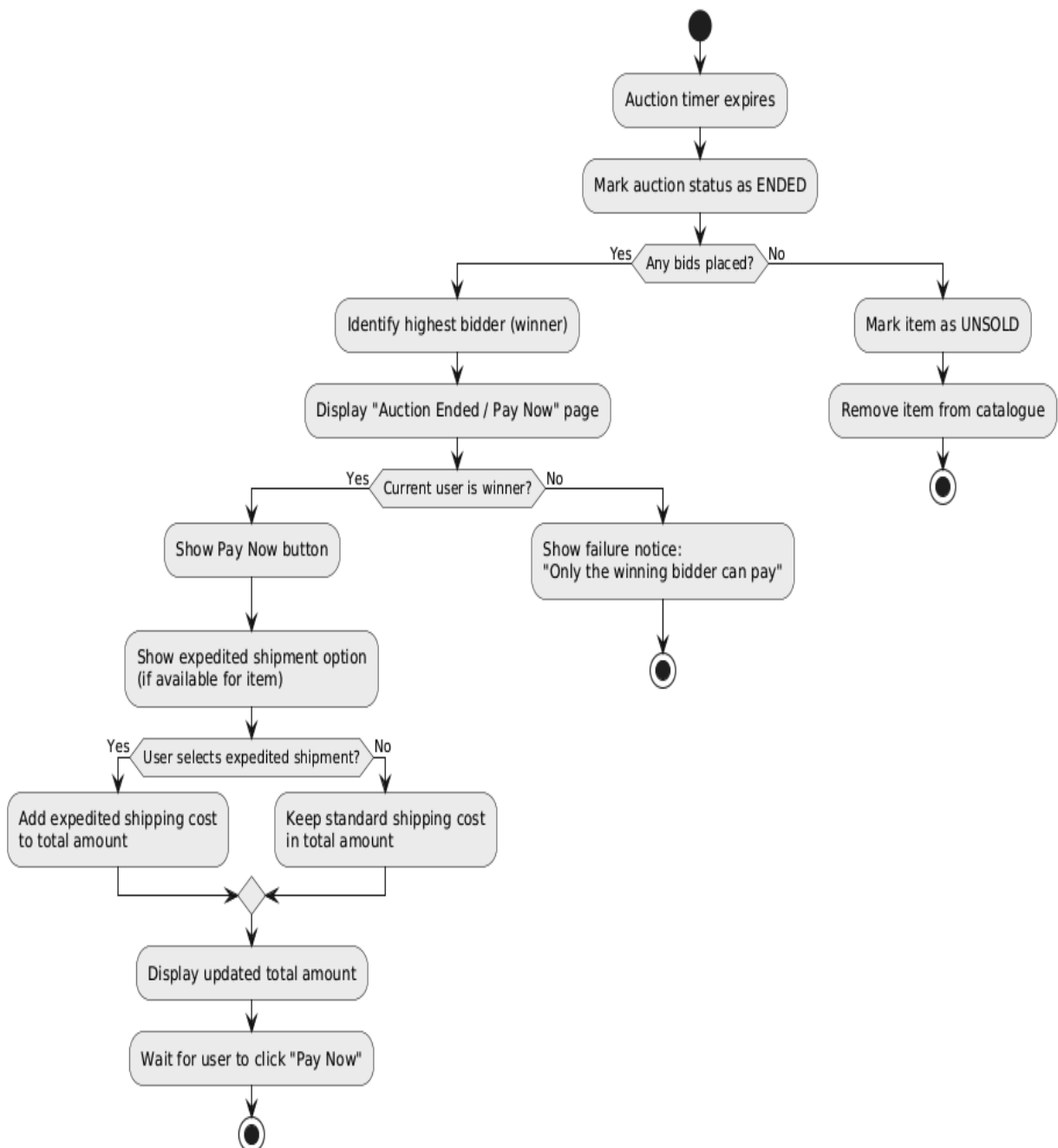
Sequence diagram for UC3:



## 5. Activity Diagrams

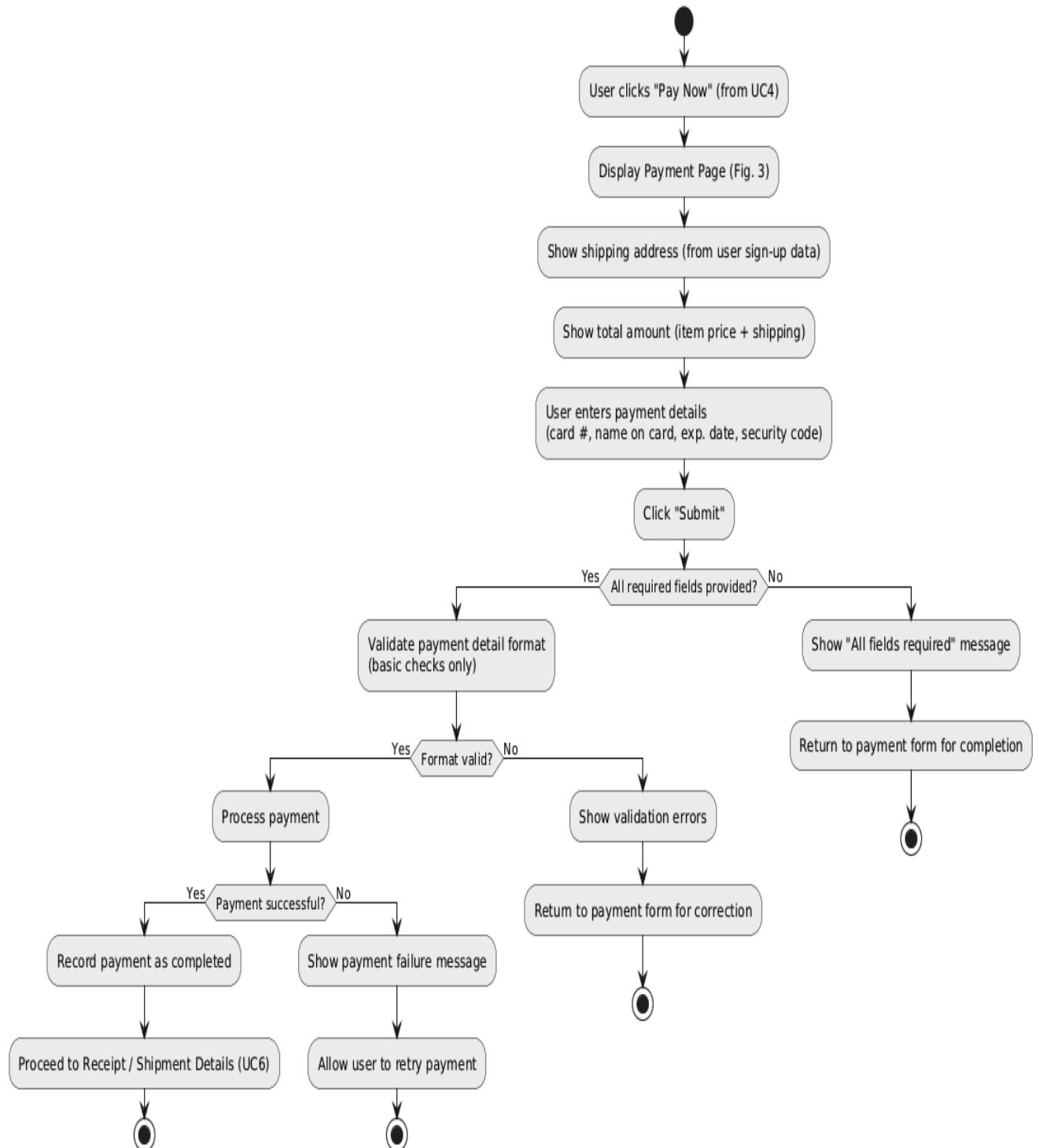
### Activity Diagram for UC4:

**AD-UC4: Auction Ended (Pay Now page + expedited option)**



## Activity Diagram for UC5:

**AD-UC5: Payment (Winner submits card details)**



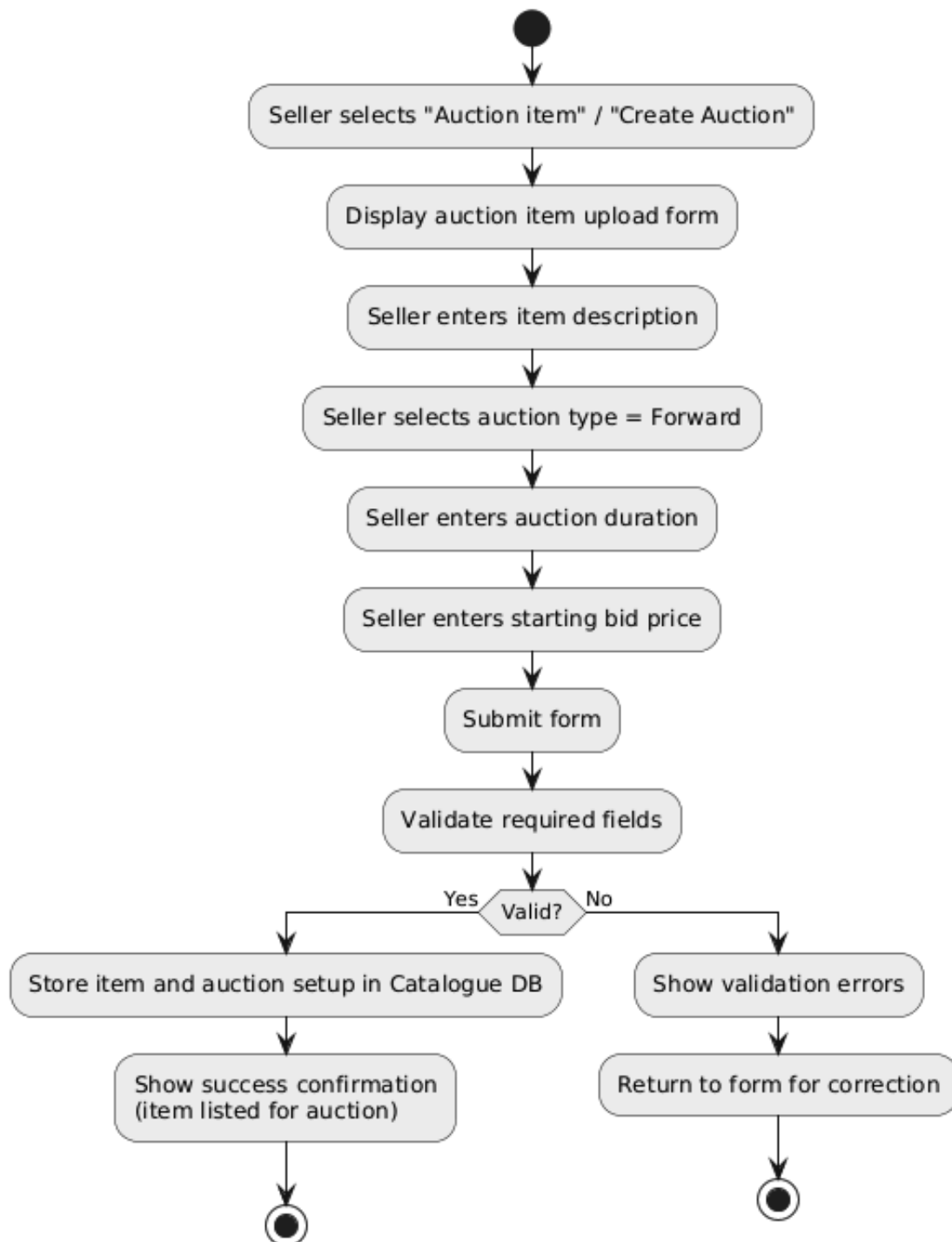
### Activity Diagram for UC6:

#### **AD-UC6: Receipt Page and Shipment Details**

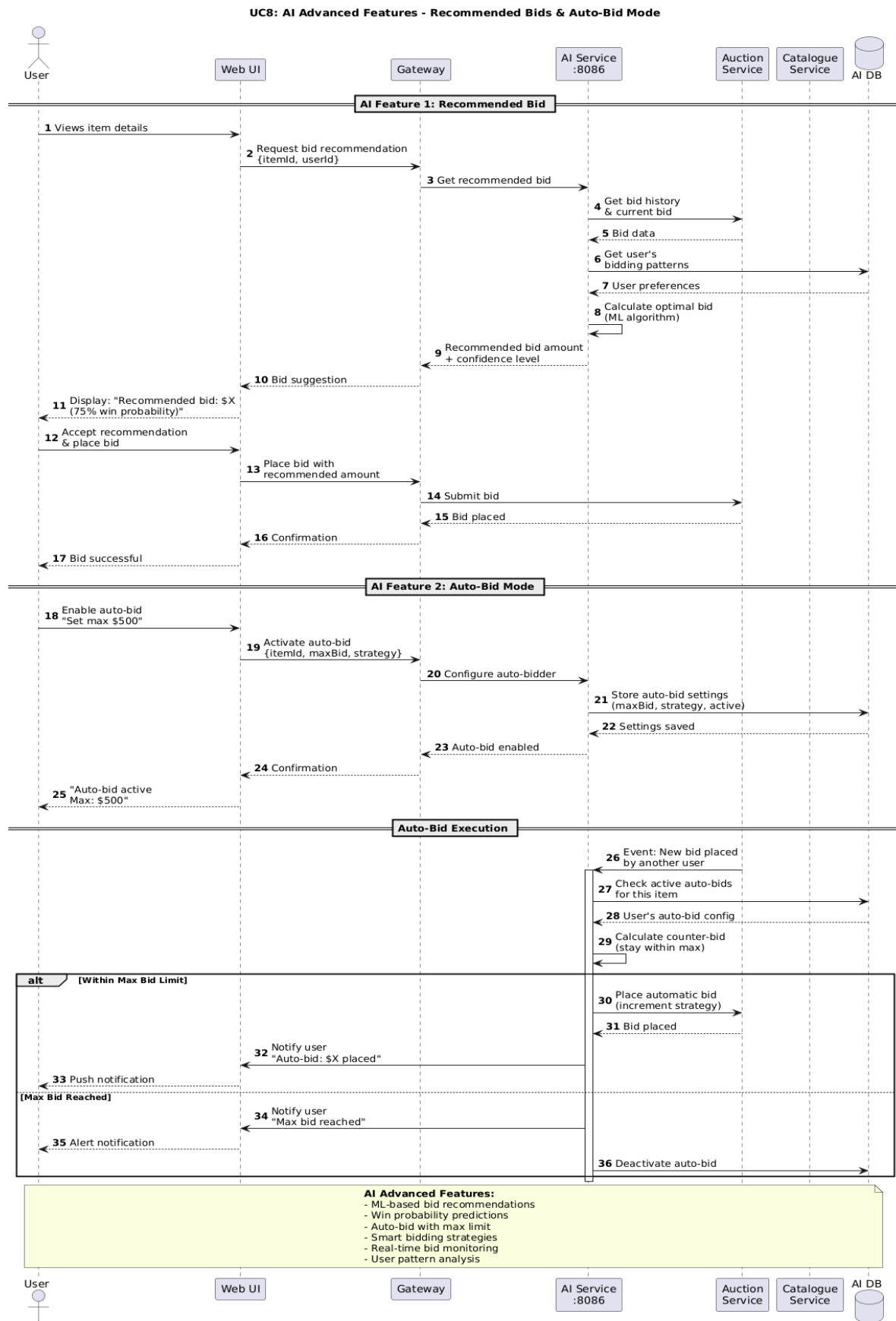


## Activity Diagram for UC7:

### **AD-UC7: Auction Item (Seller Uploads Item for Forward Auction)**



# Activity Diagram for UC8:



## 6. Architecture Overview

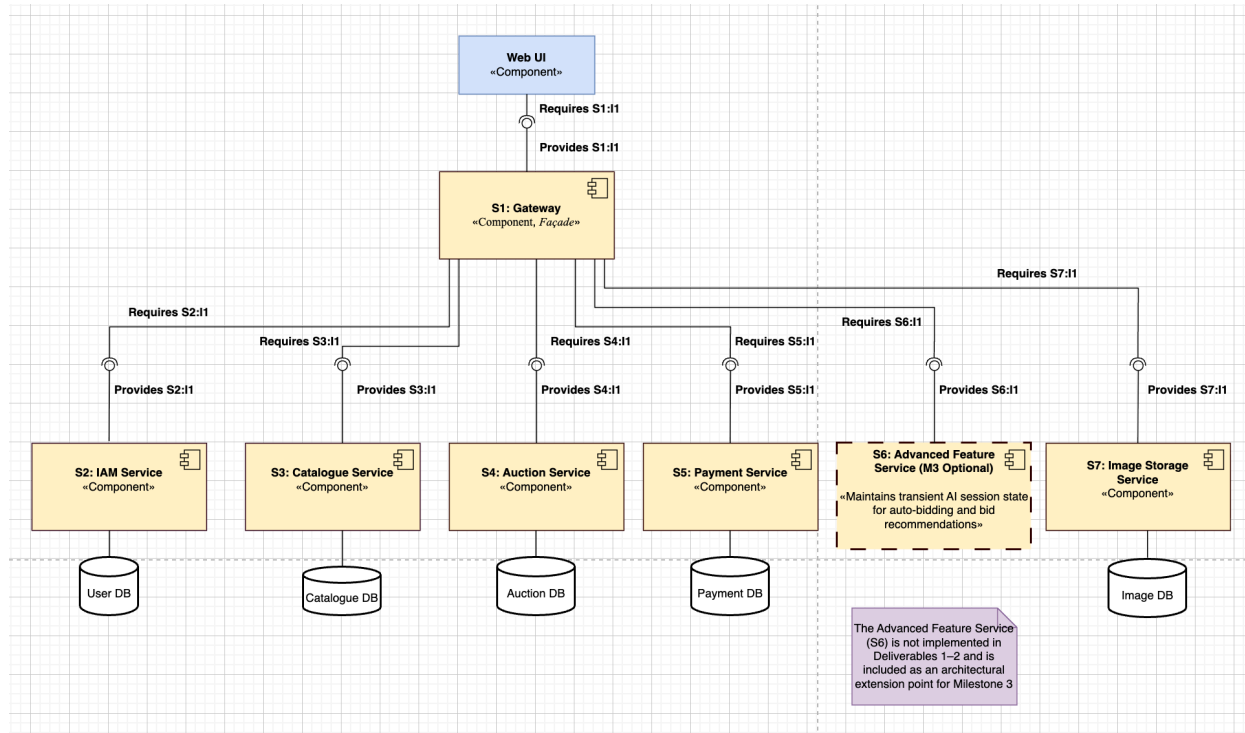
The Forward Auction System is implemented using a **layered architecture** to promote maintainability, modularity, and clarity. Each layer has a well-defined responsibility and communicates with adjacent layers through clearly defined interfaces.

### Architectural Layers:

1. **Presentation Layer:** Handles user interaction through web-based pages for signing up, signing in, browsing auctions, bidding, payment, and receipt display.
2. **Application Layer:** Coordinates incoming requests, enforces workflow logic, and mediates communication between layers. This layer includes a **Gateway (Façade) component**, which serves as a single entry point for the user interface and orchestrates use case execution across backend services.
3. **Domain Layer:** Implements the core business logic of the system, including auction rules, bid validation, auction state transitions, and payment eligibility checks.
4. **Data Layer:** Manages persistent storage for users, auctions, catalogue items, bids, and payment records through dedicated databases accessed by the corresponding services.

This architecture ensures **high cohesion within components and low coupling between layers**, enabling clear separation of concerns and simplifying future maintenance and extension.

## UML Component Diagram:



## Services Table:

| Service Name         | Description                                                                                                           | Exposed Interface Names | Interface Description                                                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S1: Gateway (Façade) | Single entry point for the Web UI. Orchestrates UC1-UC7 workflows and enforces “one selected item per login session.” | S1:I1                   | S1:I1 Gateway Façade: UI-facing interface for sign-up/sign-in, catalogue search/selection, bidding, auction-ended/pay-now, payment submission, and receipt retrieval. |
| S2: IAM Service      | Handles user registration, authentication, and                                                                        | S2:I1                   | S2:I1 IAM API: Create user, authenticate user, reset                                                                                                                  |

|                                                       |                                                                                                                                                                                                                                                                                        |       |                                                                                                                                                     |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                       | password reset. Stores user profile and shipping address in User DB.                                                                                                                                                                                                                   |       | password, and provide user shipping address.                                                                                                        |
| S3: Catalogue Service                                 | Manages auction item listings and item metadata (description, shipping cost, optional expedited shipping cost, shipping days). Supports keyword search and item retrieval.                                                                                                             | S3:I1 | Item metadata, search, visibility, shipping info, auction configuration (starting price, duration), and item lifecycle visibility (active/inactive) |
| S4: Auction Service                                   | Manages forward auction logic: current highest bid and bidder, bid validation (integer, strictly increasing), auction timer, and winner determination. Stores auction data in Auction DB.                                                                                              | S4:I1 | S4:I1 Auction API: Bidding logic, bid validation, bid history, auction timing, winner determination, auction state                                  |
| S5: Payment Service                                   | Validates required payment fields and records payment outcome. Produces receipt details after successful payment. Stores payment records in Payment DB. The Payment Service assumes the user has already been authorized as the auction winner and does not perform winner validation. | S5:I1 | S5:I1 Payment API: Submit payment and retrieve receipt data.                                                                                        |
| S6: Advanced Feature Service (Milestone 3 - Optional) | Optional Milestone 3 extension point. Provides AI-assisted / self-adaptive bidding                                                                                                                                                                                                     | S6:I1 | High-level API for starting an AI-bid assistant session and returning                                                                               |

|                           |                                                                                                                                          |       |                                                           |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------|-----------------------------------------------------------|
|                           | support (UC8-x). Not required for Deliverable 1 implementation; included as a placeholder interface.                                     |       | recommendations or auto-bid actions (Milestone 3 only).   |
| S7: Image Storage Service | Handles image upload for seller item creation. Stores images and returns URLs/IDs that are saved with item metadata in the Catalogue DB. | S7:I1 | Image upload + retrieval APIs used by Gateway during UC7. |

### **Interfaces Table:**

#### **S1:I1 - Gateway Façade (Used by Web UI):**

| Interface Name | Operations                                            | Operation Descriptions                                                                                                                          |
|----------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| S1:I1          | SignUp(userProfile) : Result (returned by Web UI)     | Creates user by forwarding data to IAM; requires username, password, first/last name, and shipping address.                                     |
|                | SignIn(username, password) : Session (used by Web UI) | Authenticates user via IAM; returns session/token for subsequent calls.                                                                         |
|                | ForgotPassword(resetInfo) : Result (used by Web UI)   | Triggers password reset flow via IAM                                                                                                            |
|                | SearchItems(keyword) : ItemList (used by Web UI)      | Returns list of matching item metadata from Catalogue; auction-specific data (current price, remaining time) is retrieved from Auction Service. |

|  |                                                                     |                                                                                                             |
|--|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
|  | SelectItem(itemId) : Result<br>(used by Web UI)                     | Stores selected item for the current login session; enforces one item per session rule.                     |
|  | GetBidPage(itemId) : BidPageDTO (used by Web UI)                    | Returns item details + current highest bid/bidder + remaining time for bidding page.                        |
|  | PlaceBid(itemId, amount:int) : BidStateDTO (used by Web UI)         | Submits bid to Auction service; validates integer and strictly increasing rule; returns updated state.      |
|  | GetAuctionEnded(itemId) : AuctionEndedDTO (used by Web UI)          | Returns auction-ended view info and winner status; includes expedited shipping option from Catalogue.       |
|  | PayNow(itemId, expedited:boolean) : Result (used by Web UI)         | Verifies winner eligibility and authorizes access to the payment flow.                                      |
|  | GetPaymentPage(itemId) : PaymentPageDTO (used by Web UI)            | Loads payment page with user shipping address (IAM) + winning price (Auction) + shipping costs (Catalogue). |
|  | SubmitPayment(itemId, paymentDetails) : ReceiptRef (used by Web UI) | Validates all payment fields are present; total = item price + shipping (+ expedited if selected).          |
|  | GetReceipt(receiptRef) : ReceiptDTO (used by Web UI)                | Returns receipt info including total paid and “ships in xxx days” shipment text.                            |

|  |                                                            |                                                                                              |
|--|------------------------------------------------------------|----------------------------------------------------------------------------------------------|
|  | UploadItem(itemInfo) : Result<br>(used by Web UI / Seller) | Creates an auction item via Catalogue with duration, starting bid, and forward auction type. |
|--|------------------------------------------------------------|----------------------------------------------------------------------------------------------|

### **S2:I1 - IAM API (Used by Gateway):**

| Interface Name | Operations                                                 | Operation Descriptions                                   |
|----------------|------------------------------------------------------------|----------------------------------------------------------|
| S2:I1          | CreateUser(userProfile) : Result<br>(used by S1)           | Stores user credentials and shipping address in User DB. |
|                | Authenticate(username, password) : AuthResult (used by S1) | Verifies credentials for sign-in.                        |
|                | ResetPassword(resetInfo) : Result<br>(used by S1)          | Updates the password for the user.                       |
|                | GetShippingAddress(userId) : AddressDTO (used by S1)       | Returns user address used on the payment page.           |

### **S3:I1 - Catalogue API (Used by Gateway):**

| Interface Name | Operations                                                                   | Operation Descriptions                                                              |
|----------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| S3:I1          | Search(keyword) : ItemList<br>(used by S1)                                   | Returns items matching keyword search.                                              |
|                | GetItemDetails(itemId) : ItemDTO (used by S1)<br>[DTO: Data Transfer Object] | Returns item description, shipping cost, expedite cost (if any), and shipping days. |
|                | CreateItem(itemInfo) : Result<br>(used by S1)                                | Adds a new forward-auction item (seller upload).                                    |

**S4:I1 - Auction API (Used by Gateway):**

| Interface Name | Operations                                                        | Operation Descriptions                                                      |
|----------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------|
| S4:I1          | GetAuctionState(itemId) : BidStateDTO (used by S1)                | Returns highest bid, highest bidder ID, and remaining time.                 |
|                | PlaceBid(itemId, bidderId, amount:int) : BidStateDTO (used by S1) | Accepts bids only if integer and strictly greater than current highest bid. |
|                | GetWinner(itemId) : WinnerDTO (used by S1)                        | After auction end, returns winning bidder and winning bid price.            |

**S5:I1 - Payment API (Used by Gateway):**

| Interface Name | Operations                                          | Operation Descriptions                                                     |
|----------------|-----------------------------------------------------|----------------------------------------------------------------------------|
| S5:I1          | SubmitPayment(paymentDTO) : ReceiptRef (used by S1) | Requires card number, name, expiry, security code; records payment status. |
|                | GetReceipt(receiptRef) : ReceiptDTO (used by S1)    | Returns total paid and receipt info for UC6.                               |

**S6:I1 - Advanced API (Milestone 3 Placeholder):**

| Interface Name | Operations                                          | Operation Descriptions                                  |
|----------------|-----------------------------------------------------|---------------------------------------------------------|
| S6:I1          | startAdvancedSession(context): SessionRef           | Starts UC8-x AI feature session (optional Milestone 3). |
|                | recommendBid(sessionRef, constraints): BidAdviceDTO | Returns recommended bid / next action                   |

|  |                                 |                                                              |
|--|---------------------------------|--------------------------------------------------------------|
|  |                                 | (UC8-x).                                                     |
|  | autoBid(sessionRef, maxBid:int) | Optional auto-bid mode up to a user-defined max bid (UC8-x). |

### **S7:I1 - Image Storage Service:**

| Interface Name | Operations                                    | Operation Descriptions                                               |
|----------------|-----------------------------------------------|----------------------------------------------------------------------|
| S7:I1          | uploadImages(itemId, files[]): List<ImageRef> | Uploads one or more images and returns stored references/URLs (UC7). |
|                | getImage(imageId): Binary/URL                 | Retrieves an image for display (UC2/UC7).                            |

## **7. Activities Plan, Product Backlog, and Sprint Backlog:**

### **7.1 Activities Plan**

Our team will follow an Agile/Scrum methodology, divided into two major Sprints (Milestones). Each sprint will last approximately 4 weeks.

- **Sprint 1 (Milestone 2):** Focuses on the "Back-End First" approach. The goal is to establish the core microservices (IAM, Catalogue, Auction, Payment), set up the SQLite databases, and implement the Gateway API. The UI will be minimal but functional to demonstrate the use cases.
- **Sprint 2 (Milestone 3):** Focuses on "Deployment & Advanced Features." The goal is to polish the UI, containerize the application using Docker, implement the AI-assisted bidding feature (UC8), and ensure real-time updates via WebSockets.

### **Sprint Backlog: Milestone 2 (Weeks 1-4)**

Selected items from Product Backlog to be completed for the next deadline:

1. **Environment Setup:** Initialize Spring Boot projects for IAM, Catalogue, Auction, Gateway.
2. **Database Design:** Implement SQLite schemas for UserDB, CatalogueDB, AuctionDB, and PaymentDB.
3. **UC1 (Sign-Up/In):** Implement IAM Service and JWT Token generation.
4. **UC2 (Browse):** Implement Catalogue Service (Search/List items).
5. **UC3 (Bidding):** Implement Auction Service (Place Bid logic, validation).
6. **UC4 (Auction End):** Implement timer logic to mark auctions as ENDED.
7. **UC5 & UC6 (Payment/Receipt):** Implement Payment Service and basic receipt generation.
8. **UC7 (Seller Upload):** Allow item creation via API.
9. **Integration:** Connect Gateway to backend services using REST Template/Feign.

### Sprint Backlog: Milestone 3 (Weeks 5-8)

Selected items for the final release:

1. **Containerization:** Create Dockerfiles for all services and docker-compose.
2. **UC8 (Advanced AI):** Implement the Advanced Feature Service for bid recommendations.
3. **UI Polish:** Improve CSS/Styling for a cohesive user experience.
4. **Real-time Updates:** Implement WebSockets for live bid updates (push notifications).
5. **Testing:** Comprehensive integration testing and bug fixes.

## 7.2 Product Backlog

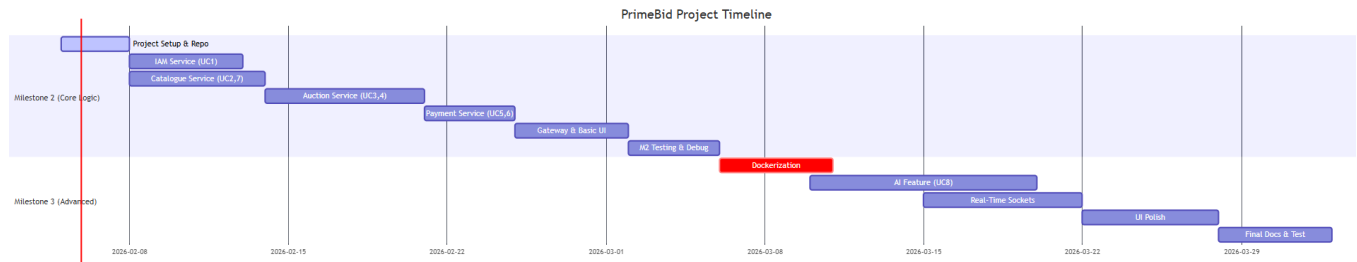
| ID | Description                                                                       | Use Case | Priority | Milestone |
|----|-----------------------------------------------------------------------------------|----------|----------|-----------|
| 1  | As a user, I want to sign up and sign in to securely access the system            | UC1      | High     | M2        |
| 2  | As a bidder, I want to search and browse items so I can find products for auction | UC2      | High     | M2        |
| 3  | As a bidder, I want to place increasing integer bids on items                     | UC3      | High     | M2        |
| 4  | As a winner, I want to be notified                                                | UC4      | Medium   | M2        |

|    |                                                                                                                                                                |     |        |    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|--------|----|
|    | when the auction ends so I can pay for my item                                                                                                                 |     |        |    |
| 5  | <b>As a winner</b> , I want to be able to select expedited shipping                                                                                            | UC4 | Medium | M2 |
| 6  | <b>As a winner</b> , I want to submit my payment details to complete my purchase                                                                               | UC5 | High   | M2 |
| 7  | <b>As a winner</b> , I want a receipt and shipment timeline after payment                                                                                      | UC6 | Medium | M2 |
| 8  | <b>As a seller</b> , I want to upload item details and set auction durations                                                                                   | UC7 | High   | M2 |
| 10 | <b>As a user</b> , I want to use AI-assisted bidding features (recommended bids and optional auto-bidding) to help me participate more effectively in auctions | UC8 | Low    | M3 |
| 11 | <b>As a bidder</b> , I want my bid to be rejected if it is not an integer or not higher than the current bid                                                   | UC3 | High   | M2 |
| 12 | <b>As the system</b> , I want to enforce that only one item can be selected per login session                                                                  | UC2 | High   | M2 |
| 13 | <b>As the system</b> , I want to automatically end auctions when their duration expires                                                                        | UC4 | High   | M2 |

|    |                                                                                                  |           |        |    |
|----|--------------------------------------------------------------------------------------------------|-----------|--------|----|
| 14 | <b>As the system,</b> I want to determine whether an auction is sold or unsold when it ends      | UC4       | Medium | M2 |
| 15 | <b>As the system,</b> I want to remove unsold or completed items from the active catalogue       | UC4       | Medium | M2 |
| 16 | <b>As the system,</b> I want to restrict payment access to the winning bidder only               | UC5       | High   | M2 |
| 17 | <b>As a user,</b> I want clear error messages when an action is invalid or not permitted         | UC1-UC6   | Medium | M2 |
| 18 | <b>As the system,</b> I want to securely store and manage user sessions after login              | UC1       | High   | M2 |
| 19 | <b>As a seller,</b> I want to upload images for auction items                                    | UC7       | Medium | M2 |
| 20 | <b>As the system,</b> I want to persist all auction, bid, and payment data reliably              | All       | High   | M2 |
| 21 | <b>As a developer,</b> I want to define and execute test cases to validate system behavior (TDD) | All       | High   | M2 |
| 22 | <b>As a user,</b> I want the system to log auction and payment outcomes                          | UC4 - UC6 | Low    | M2 |

|    |                                                                                     |     |     |    |
|----|-------------------------------------------------------------------------------------|-----|-----|----|
|    | for audit purposes                                                                  |     |     |    |
| 23 | <b>As a user</b> , I want the system to support AI-assisted bidding recommendations | UC8 | Low | M3 |

## 7.3 GANTT Diagram



## 7.4 Group Meeting Logs

|                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Meeting Date     | Jan 23, 2026                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Attendance       | Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)                                                                                                                                                                                                                                                                                                                                                                                         |
| Topics Discussed | <ol style="list-style-type: none"><li>Review of Project Description and "PrimeBid" requirements.</li><li>Analysis of the 7 Use Cases provided in the instructions.</li><li>Discussion on High-Level Architecture (Microservices vs. Layered/Monolithic).</li><li>Selection of technologies for the implementation phase.</li></ol>                                                                                                                                           |
| Decisions Made   | <p><b>Architecture:</b> Decided to use a <b>Layered Architecture</b> for Deliverable 1 to simplify the initial design, while keeping interfaces loose for a potential Microservices migration in Milestone 2.</p> <p><b>Use Case Split:</b> Agreed to map <b>UC1, UC2, UC3</b> for Sequence Diagrams and <b>UC4, UC5, UC6, UC7</b> for Activity Diagrams to ensure no overlap.</p> <p><b>Tools:</b> Selected Draw.io for UML modeling and Google Docs for collaboration.</p> |

|                         |                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Task Assignments</b> | <p><b>All Members:</b> Read the Marking Schema and Deliverable Instructions in detail.</p> <p><b>Nathan:</b> Initialize the Design Document structure and draft the Introduction/Overview.</p> <p><b>Umer:</b> Begin drafting initial sketches for Sequence Diagrams (UC 1-3).</p> <p><b>Imaduddin:</b> Research Controller/Service/Repository patterns for the Component Diagram.</p> |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                         |                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Meeting Date</b>     | <b>Jan 30, 2026</b>                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Attendance</b>       | Nathan Binu Edappilly, Umer Shaikh, Imaduddin Ahmed, Pawanjot Randhawa (All Present)                                                                                                                                                                                                                                                                                                          |
| <b>Topics Discussed</b> | <ol style="list-style-type: none"> <li>1. Review of initial Sequence and Activity diagram drafts.</li> <li>2. Defining the API Interfaces (REST endpoints) for the Architecture section.</li> <li>3. Planning the Product Backlog and GANTT chart timeline.</li> <li>4. TDD Strategy: Brainstorming edge cases for the required 16 tests.</li> </ol>                                          |
| <b>Decisions Made</b>   | <p><b>Interface Definition:</b> Finalized the "Auction Service" and "Catalogue Service" split. Agreed that the Gateway will handle session management.</p> <p><b>TDD Rule:</b> Confirmed that each member must write exactly 4 test cases to meet the assignment requirement.</p> <p><b>Workload Division:</b> Finalized the specific section ownership for the final report compilation.</p> |

|                         |                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Task Assignments</b> | <p><b>Umer:</b> Finalize all Sequence and Activity Diagrams (clean up notation).</p> <p><b>Nathan:</b> Complete the Architecture Section (Component Diagram + Interface Tables).</p> <p><b>Pawanjot:</b> Create the Project Plan (GANTT/Backlog) and compile the final TDD table.</p> <p><b>Imaduddin:</b> Integrate all sections, write the AI Acknowledgment, and perform final proofreading.</p> |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 8. Test Driven Development:

|                       |                                                                                                                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-01                                                                                                                                                                                                     |
| Category              | Authentication / User Registration                                                                                                                                                                            |
| Requirements Coverage | UC1.1-SignUp-Success                                                                                                                                                                                          |
| Initial Condition     | System running; User DB available; username not already used.                                                                                                                                                 |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens Sign-Up page.</li> <li>2. User enters username, password, first name, last name, and shipping address.</li> <li>3. User clicks <b>Submit</b>.</li> </ol> |
| Expected Outcome      | Account is created; user record stored in User DB; success message displayed; user can proceed to Sign-In.                                                                                                    |
| Notes                 | Password should be stored securely (hashed), not plain text.                                                                                                                                                  |

|                       |                                                                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-02                                                                                                                                                                                  |
| Category              | Authentication / User Registration                                                                                                                                                         |
| Requirements Coverage | UC1.1-SignUp-Validation-RequiredFields                                                                                                                                                     |
| Initial Condition     | System running; User DB available.                                                                                                                                                         |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens Sign-Up page.</li> <li>2. User leaves one required field blank (e.g., postal code).</li> <li>3. User clicks <b>Submit</b>.</li> </ol> |
| Expected Outcome      | Validation error shown; no user record created.                                                                                                                                            |
| Notes                 | Verify server-side validation, not UI-only.                                                                                                                                                |

|                       |                                                                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-03                                                                                                                                                                                       |
| Category              | Authentication / User Registration                                                                                                                                                              |
| Requirements Coverage | UC1.1-SignUp-Username-Unique                                                                                                                                                                    |
| Initial Condition     | System running; User DB contains username “nathan123”.                                                                                                                                          |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens Sign-Up page.</li> <li>2. User enters username “nathan123” with valid remaining details.</li> <li>3. User clicks <b>Submit</b>.</li> </ol> |
| Expected Outcome      | Error message “username already exists”; no duplicate user created.                                                                                                                             |
| Notes                 | Ensure uniqueness enforced at DB level.                                                                                                                                                         |

|                       |                                                                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-04                                                                                                                                                                  |
| Category              | Authentication / Sign-In                                                                                                                                                   |
| Requirements Coverage | UC1.2-SignIn-Success                                                                                                                                                       |
| Initial Condition     | System running; valid user exists in User DB.                                                                                                                              |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens Sign-In page.</li> <li>2. User enters valid username and password.</li> <li>3. User clicks <b>Sign In</b>.</li> </ol> |
| Expected Outcome      | User is authenticated; session established; user redirected to Catalogue page.                                                                                             |

|       |                                                          |
|-------|----------------------------------------------------------|
| Notes | Session/token should be created and stored consistently. |
|-------|----------------------------------------------------------|

|                       |                                                                                                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-05                                                                                                                                                                          |
| Category              | Authentication / Sign-In                                                                                                                                                           |
| Requirements Coverage | UC1.2-SignIn-InvalidCredentials                                                                                                                                                    |
| Initial Condition     | System running; user exists in User DB.                                                                                                                                            |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens Sign-In page.</li> <li>2. User enters correct username but wrong password.</li> <li>3. User clicks <b>Sign In</b>.</li> </ol> |
| Expected Outcome      | “Invalid credentials” message displayed; no session created.                                                                                                                       |
| Notes                 | Avoiding leak to which part was wrong.                                                                                                                                             |

|                       |                                                                                                                                                                                                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-06                                                                                                                                                                                                                                                                          |
| Category              | Authentication / Forgot Password                                                                                                                                                                                                                                                   |
| Requirements Coverage | UC1.2-ForgotPassword-ResetSuccess                                                                                                                                                                                                                                                  |
| Initial Condition     | System running; user exists; password reset enabled.                                                                                                                                                                                                                               |
| Procedure             | <ol style="list-style-type: none"> <li>1. User clicks <b>Forgot Password</b>.</li> <li>2. User provides required reset input (e.g., username / token flow).</li> <li>3. User enters a new password and submits reset.</li> <li>4. User signs in using the new password.</li> </ol> |

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| Expected Outcome | Password updated in User DB; user can sign in with new password. |
| Notes            | Ensure old password no longer works.                             |

|                       |                                                                                                                               |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-07                                                                                                                     |
| Category              | Catalogue / Search                                                                                                            |
| Requirements Coverage | UC2.1-ItemSearch-Keyword                                                                                                      |
| Initial Condition     | System running; Catalogue DB contains items with keywords (e.g., “watch”, “vintage”).                                         |
| Procedure             | <ol style="list-style-type: none"> <li>1. User enters keyword in search bar.</li> <li>2. User clicks <b>Search</b></li> </ol> |
| Expected Outcome      | Matching items returned and displayed.                                                                                        |
| Notes                 | Search should use keywords stored in Catalogue DB.                                                                            |

|                       |                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-08                                                                                                                  |
| Category              | Catalogue / Display Auctioned Items                                                                                        |
| Requirements Coverage | UC2.2-DisplayAuctionedItems-Fields                                                                                         |
| Initial Condition     | System running; at least one auctioned item exists.                                                                        |
| Procedure             | <ol style="list-style-type: none"> <li>1. User browses catalogue results.</li> <li>2. User views item listings.</li> </ol> |
| Expected Outcome      | Each displayed item shows: full name, current                                                                              |

|       |                                                                            |
|-------|----------------------------------------------------------------------------|
|       | bidding price, auction type (Forward), and remaining time (if applicable). |
| Notes | Forward auction type should be explicit.                                   |

|                       |                                                                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-09                                                                                                                                                      |
| Category              | Session / Item Selection                                                                                                                                       |
| Requirements Coverage | UC2.3-ItemSelection-OnePerSession                                                                                                                              |
| Initial Condition     | System running; user is logged in; multiple items displayed.                                                                                                   |
| Procedure             | <ol style="list-style-type: none"> <li>1. User selects Item A using radio button.</li> <li>2. User attempts to select Item B in same login session.</li> </ol> |
| Expected Outcome      | System enforces <b>only one selected item per login session</b> (either prevents second selection or replaces selection in a controlled way).                  |
| Notes                 | Align behavior with your Gateway rule; document which approach you implement.                                                                                  |

|                       |                                                                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-10                                                                                                                                          |
| Category              | Auction / Place Bid                                                                                                                                |
| Requirements Coverage | UC3-Bid-ValidIncreasingInteger                                                                                                                     |
| Initial Condition     | System running; auction active; current highest bid = 25.                                                                                          |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens selected item bidding page.</li> <li>2. User enters bid = 30 (integer, higher than</li> </ol> |

|                  |                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------|
|                  | 25).<br><br>3. User clicks <b>BID</b> .                                                                   |
| Expected Outcome | Bid accepted; Auction DB updated with new highest bid and highest bidder; UI refresh shows updated price. |
| Notes            | Must be strictly higher, and integer only.                                                                |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-11                                                                                                                                                                                                                                                                                                                                                                                                             |
| Category              | Auction / Bid Validation                                                                                                                                                                                                                                                                                                                                                                                              |
| Requirements Coverage | UC3-Bid-RejectNotIncreasing                                                                                                                                                                                                                                                                                                                                                                                           |
| Initial Condition     | System running; user is logged in; an auction is ACTIVE for selected Item X; current highest bid = <b>25</b> ; user has selected Item X in this login session.                                                                                                                                                                                                                                                        |
| Procedure             | <ol style="list-style-type: none"> <li>1. User opens selected item bidding page for Item X.</li> <li>2. User enters bid = <b>25</b> (equal to current highest).</li> <li>3. User clicks <b>BID</b>.</li> <li>4. User enters bid = <b>20</b> (lower than current highest).</li> <li>5. User clicks <b>BID</b>.</li> <li>6. User enters bid = <b>25.5</b> (non-integer).</li> <li>7. User clicks <b>BID</b>.</li> </ol> |
| Expected Outcome      | <ul style="list-style-type: none"> <li>● Bid = 25 is rejected with an error indicating the bid must be <b>strictly higher</b> than the current highest bid.</li> <li>● Bid = 20 is rejected with an error indicating the bid must be <b>strictly higher</b> than the current</li> </ul>                                                                                                                               |

|       |                                                                                                                                                                                                                                                                                                                             |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | <p>highest bid.</p> <ul style="list-style-type: none"> <li>● Bid = 25.5 is rejected with an error indicating bids must be <b>integers only</b>.</li> <li>● <b>No changes</b> occur in Auction DB: highest bid remains 25, highest bidder unchanged, and no new bid-history record is inserted for rejected bids.</li> </ul> |
| Notes | Enforce server-side bid validation to prevent users bypassing rules via client-side manipulation. The auction state remains unchanged for invalid bids.                                                                                                                                                                     |

|                       |                                                                                                                                                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-12                                                                                                                                                                                                           |
| Category              | Auction / State Management                                                                                                                                                                                          |
| Requirements Coverage | UC4-AuctionEnded-UnsoldItem                                                                                                                                                                                         |
| Initial Condition     | Auction for item X has N time left; No bid have been placed; N = 1 second                                                                                                                                           |
| Procedure             | <ol style="list-style-type: none"> <li>1. Auction ends after N amount of time</li> <li>2. Auction service trigger getWinner logic</li> <li>3. Observe status of item X in Auction and catalogue database</li> </ol> |
| Expected Outcome      | In auction database auction should be marked as ENDED and item should be marked as UNSOLD; In catalogue database item should be removed;                                                                            |
| Notes                 | Unsold items should be removed to avoid cluttering the catalogue database; Auction details can stay as the auction owner should be able to see the status of the auction                                            |

|          |                         |
|----------|-------------------------|
| TestID   | TDD-TC-13               |
| Category | Payment / Authorization |

|                       |                                                                                                                                              |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Requirements Coverage | UC4-PaymentAccess-WinnersOnly                                                                                                                |
| Initial Condition     | Auction ended; User is logged in; User did not win the auction                                                                               |
| Procedure             | <ol style="list-style-type: none"> <li>1. User click on the item page for the completed auction</li> <li>2. User clicks “Pay Now”</li> </ol> |
| Expected Outcome      | UI displays a failure notice indicating that only the winning user is authorized to proceed to payment.                                      |
| Notes                 | Gateway has to verify the user ID against the winner ID stored in the database.                                                              |

|                       |                                                                                                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-14                                                                                                                                                          |
| Category              | Payment / Shipping Logic                                                                                                                                           |
| Requirements Coverage | UC4-ExpeditedShipping-Calculation                                                                                                                                  |
| Initial Condition     | User if logged in; User has won the auction; User has clicked pay now and has been redirected to the payment screen; Winning bid =\$100, Expedited Shipping = \$25 |
| Procedure             | <ol style="list-style-type: none"> <li>1. User selects "expedite shipping” option</li> <li>2. User can observe updated total amount</li> </ol>                     |
| Expected Outcome      | UI should display a total of \$125                                                                                                                                 |
| Notes                 | Expedited cost should be pulled from the database for each specific item.                                                                                          |

|                       |                             |
|-----------------------|-----------------------------|
| TestID                | TDD-TC-15                   |
| Category              | Payment / Input Validation  |
| Requirements Coverage | UC5-Payment-FieldValidation |

|                   |                                                                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Initial Condition | User is logged in; User has won auction and has navigated to payment details page                                                                                                      |
| Procedure         | <ol style="list-style-type: none"> <li>1. User enters Name, Card Number, Expiration</li> <li>2. User does not fill out security code field</li> <li>3. User clicks “Submit”</li> </ol> |
| Expected Outcome  | The system displays a validation error indicating that all fields must be filled in.                                                                                                   |
| Notes             | System should also check proper formatting for each field, so not character in card number; Transaction should not be recorded is validation error occurs                              |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-16                                                                                                                                                                                                                                                                                                                                                                                      |
| Category              | Receipt / Shipping Details                                                                                                                                                                                                                                                                                                                                                                     |
| Requirements Coverage | UC6-ReceiptPage-ShowTotalsAndShipping                                                                                                                                                                                                                                                                                                                                                          |
| Initial Condition     | System running; user is logged in; user is the <b>winning bidder</b> for Item X; payment for Item X is <b>COMPLETED</b> in Payment DB; Catalogue DB contains Item X shipping timeline text (e.g., “Ships in 5 days”).                                                                                                                                                                          |
| Procedure             | <ol style="list-style-type: none"> <li>1. After payment success, user is redirected to the <b>Receipt &amp; Shipping Details</b> page (UC6).</li> <li>2. System retrieves transaction details (transactionId, winningBid, shipping cost, total paid).</li> <li>3. System retrieves item shipping info from Catalogue DB (e.g., “Ships in 5 days”).</li> <li>4. User views the page.</li> </ol> |

|                  |                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Expected Outcome | <ul style="list-style-type: none"> <li>• Receipt page displays: <b>transaction ID, item name, winning price, shipping cost, and total paid.</b></li> <li>• Shipping section displays a timeline statement such as: <b>“Item will be shipped in X days”</b> (text is acceptable as per instructions).</li> <li>• The page provides a navigation option such as <b>Back to Main / Return Home.</b></li> </ul> |
| Notes            | Shipping timelines are displayed as descriptive text rather than computed calendar dates, in accordance with the project requirements. The receipt page is read-only and reflects only finalized transaction and shipment information.                                                                                                                                                                      |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-17                                                                                                                                                                                                                                                                                                                                                                                        |
| Category              | Seller / Auction Item Upload                                                                                                                                                                                                                                                                                                                                                                     |
| Requirements Coverage | UC7-CreateAuctionItem-Success                                                                                                                                                                                                                                                                                                                                                                    |
| Initial Condition     | System running; seller user is logged in; Catalogue DB and Auction DB available; image upload mechanism (Most possibly) available.                                                                                                                                                                                                                                                               |
| Procedure             | <ol style="list-style-type: none"> <li>1. Seller clicks <b>Create Auction.</b></li> <li>2. Seller enters valid item details: name, description, keywords, starting price, auction duration, shipping info, and auction type = <b>Forward.</b></li> <li>3. Seller uploads at least one image (if your implementation supports images).</li> <li>4. Seller clicks <b>Submit/Create.</b></li> </ol> |

|                  |                                                                                                                                                                                                                                                                                                                                                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  |                                                                                                                                                                                                                                                                                                                                                                      |
| Expected Outcome | <ul style="list-style-type: none"> <li>Item is created in <b>Catalogue DB</b> with status <b>ACTIVE</b> and searchable keywords stored.</li> <li>A corresponding auction record is created in <b>Auction DB</b> with start time, end time, starting bid, and status <b>ACTIVE</b>.</li> <li>UI displays a success message with a generated <b>ItemID</b>.</li> </ul> |
| Notes            | This test confirms that a seller can successfully create a new auction and that the item becomes visible for browsing and bidding. Image upload can be treated as optional or mocked if not fully implemented in this milestone.                                                                                                                                     |

|                       |                                                                                                                                                                                                                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TestID                | TDD-TC-18                                                                                                                                                                                                                                                                                                                             |
| Category              | Seller / Auction Item Upload Validation                                                                                                                                                                                                                                                                                               |
| Requirements Coverage | UC7-CreateAuctionItem-ValidationFailure                                                                                                                                                                                                                                                                                               |
| Initial Condition     | System running; seller user is logged in; Catalogue DB and Auction DB available.                                                                                                                                                                                                                                                      |
| Procedure             | <ol style="list-style-type: none"> <li>Seller clicks <b>Create Auction</b>.</li> <li>Seller enters invalid values (choose at least one): <ul style="list-style-type: none"> <li>starting price = <b>0</b> or <b>negative</b></li> <li>auction duration outside allowed range (e.g., 0 minutes or &gt; 30 days)</li> </ul> </li> </ol> |

|                  |                                                                                                                                                                                                                                       |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | <ul style="list-style-type: none"> <li>● missing required fields (e.g., name or keywords empty)</li> </ul> <p>3. Seller clicks <b>Submit/Create</b>.</p>                                                                              |
| Expected Outcome | <ul style="list-style-type: none"> <li>● Validation error is displayed describing the invalid field(s).</li> <li>● <b>No item</b> record is created in Catalogue DB and <b>no auction</b> record is created in Auction DB.</li> </ul> |
| Notes            | Enforce server-side validation for business rules, not UI-only.                                                                                                                                                                       |

## **9. Acknowledgment (AI Usage):**

We utilized Generative AI tools (specifically Large Language Models) to assist in the drafting and organization of this design document. The AI was used primarily as a productivity tool to streamline the documentation process and provide an organizational understanding of the system's execution. Specific areas where AI assistance was utilized include:

- **Test Case Generation:** We used AI to brainstorm potential edge cases and generate the initial list of Test Driven Development (TDD) scenarios (Section 8) to ensure comprehensive requirements coverage.
- **Implementation Strategy & Logic Mapping:** The AI was used to help us gain an organizational understanding of how to plan out the implementation of the project. This included identifying logical dependencies between system components to ensure our diagrams accurately reflected a coherent workflow.
- **Project Planning & Scheduling:** AI was used to help structure the Activities Plan and organize the sequential data required for the GANTT chart (Section 7), ensuring the development timeline was realistic and logically ordered.

### **Critical Evaluation of Generative AI**

As required by the project specifications, the team conducted a critique of the GenAI tools used based on our experience during this deliverable:

#### **Strengths:**

- **Rapid Prototyping of Documentation:** AI significantly reduced the time required to format complex tables (such as the TDD and Service Interface tables) and helped in drafting the initial structure of the document.
- **Edge Case Discovery:** The AI was effective at suggesting "negative" test scenarios (e.g., bidding after an auction has ended or entering non-integer values) that ensured our TDD section was robust.
- **Synthesized Planning:** It helped translate our high-level deadlines into a structured sequence of tasks suitable for a GANTT diagram.

#### **Weaknesses:**

- **Inconsistent UML Notation:** We observed that the AI frequently suggested diagram logic or labels that did not strictly adhere to standard UML notation or the specific architectural constraints of our system. For instance, initial suggestions for component connections often ignored the requirement for a Gateway/Facade.
- **Technical Hallucinations:** On several occasions, the AI proposed REST endpoints or data types that were inconsistent with our established "PrimeBid" database schema, requiring manual correction to ensure consistency across the Sequence and Architecture diagrams.
- **Generic Outputs:** The initial AI-generated descriptions were often too general; the team had to perform significant manual refining to ground the content in the specific "Forward Auctioning" business rules required for EECS 4413.

**Disclaimer:** All content generated by AI was critically reviewed, verified, and refined by the team members to ensure it accurately reflects our architectural decisions and meets the specific constraints of the project requirements.